

Anleitung für Github

Wichtige Befehle - Übersicht

Ordnerwechsel / Ordneranwahl über Windows Power Shell

```
```bash
```

## Terminal öffnen

```
cd dsp ```
```

Aktualisierungen abfragen

**Wichtig:** \ Für diese und alle weiteren Befehle muss man sich im Verzeichnis dsp befinden!

Branch auschecken

```
bash git checkout {branchname}
```

Lokalen Branch aktualisieren

```
```bash git fetch origin git pull {branchname}/origin # wobei hier origin alternativ ist, also nicht unbedingt angegeben werden muss
```

alternativ in einer Zeile unter Windows

```
git fetch origin; git pull {branchname}/origin
```

oder unter Linux

```
git fetch origin && git pull {branchname}/origin ```
```

Remote Branch aktualisieren

Achtung: \ Nur den eigenen Remote Branch aktualisieren!!!

```
```bash git status #
```

  
um sicher zu stellen, dass man auf dem eigenen Branch ist  

```
git add . #
```

 fügt alle geänderten Dateien und Verzeichnisse ausgehend vom aktuellen Verzeichnis dazu (sofern nicht in .gitignore ausgeschlossen)

es ist auch möglich, nur einzelne Dateien hinzuzufügen

`git add {Datei 1} {Datei 2} ... #` Dateien werden relativ zum aktuellen Verzeichnis angegeben und durch Leerzeichen getrennt

alternativ in einer Zeile unter Windows

`git status; git add .`

oder unter Linux

`git status && git add . ```

Neuen lokalen Branch erstellen, um Sachen zu testen

```bash`

Neuen Branch MIT den neuen Dateiänderungen erstellen

`git checkout -b '{neuer Branchname}' #` Argument -b erzeugt einen neuen Branch mit dem angegebenen Name im lokalen Repository

Neuen Branch erstellen und Änderungen VERWERFEN

`git ceckout -B '{neuer Branchname}' #` Argument -B erzeugt ebenfalls einen neuen Branch, setzt aber die aktuellen Dateiänderungen im aktuellen Branch zurück! `` Beide Befehle bewirken das automatische Wechseln in den neu erzeugten Branch.

Befehle für Fortgeschrittene Anwender / Admins

Rebase

Bei einem Rebase wird der aktuelle Branch auf den neuesten Stand aktualisiert unter Beibehaltung der Commit-Historie. Aus diesem Grund ist nach einem Rebase in einem lokalen Branch immer ein FORCE-Push erforderlich. ``bash `git rebase origin/{Branchname dessen Änderungen in den eigenen Branch übernommen werden sollen}`

wenn Konflikte vorhanden sind, diese beheben und mit git
add {Konfliktdatei 1 } {Konfliktdatei 2 } ... hinzufügen

nach der durchgeführten Aktualisierung und der Behebung
aller eventueller Konflikte

```
git push --force ``
```

Merge

Bei einem Merge wird wie beim Rebase der aktuelle Branch auf den neuesten Stand des angegebenen Branches aktualisiert, aber die Commit-Historie des eingefügten Branches wird überschrieben und geht unwiederbringlich verloren! Der Vorteil ist allerdings, dass auf diese Weise nur ein neuer Commit im aktuellen Branch erzeugt wird und daher kein force-Push erforderlich ist. Es wird also so getan, als wären die von dem anderen Branch eingefügten Änderungen im aktuellen Branch erzeugt worden.

Bestimmten Commit / bestimmte Änderung in seinen Branch übernehmen

```
`` bash git checkout {Branch mit der gewünschten Änderung} git log
```

Log-Eintrag mit der gewünschten Änderung suchen und Commit-ID kopieren

```
git switch - # bewirkt das Zurückwechseln in den vorherigen Branch git cherry-pick  
{kopierte Commit-ID} ``
```

Bei allen Befehlen ist unbedingt die Groß- und Kleinschreibung zu beachten;
auch bei den Namen der Branches. Github unterscheidet also auch zwischen
frank und Frank als Branchnamen!

Weiterführende Links

Informationen zu git und Github